In [1]:
```
import pandas as pd
import numpy as np

%load_ext autoreload
%autoreload 2
import tme6

# on définit le tableau des mois dans tme6
print(tme6.months)
```

```
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
```

TME 6- Simulation de microfermes en maraîchage diversifié

Dans les microfermes en maraîchage diversifié, les produits récoltés sont souvent vendus en direct aux particuliers sous forme de paniers hebdomadaires. Il faut pour satisfaire la clientèle avoir suffisament de sortes de légumes et suffisament de sortes de légume pour chaque catégorie. Dans la suite, on considère 8 catégories de légume. Le maraîcher choisit donc parmi les **cycles de culture** ceux qui sont adaptés pour respecter les contraintes de quantité et de diversité. La viabilité d'une microferme respectant ces contraintes sera évaluées sur la base du revenu et de la charge de travail qu'elle génère. Le but du projet est de simuler des microfermes de manière aléatoire et d'évaluer leur viabilité.

Le but de ce TME est de se donner les moyens d'étudier la viabilité de tels microfermes. Pour se faire, il nous faudrait un échantillon de microfermes sur lequel faire nos calculs statistiques (rendements moyens, etc.). Mais nous n'avons pas cet échantillon.

Nous allons donc le créer de notre mieux en utilisant un certain nombres d'informations qui nous permettrons de proposer des microfermes raisonnalbes.

On dispose de deux bases de données issues de la thèse de Kevin Morel qui rassemble les plus de 1000 cycles de culture et qui donne les critères de quantité et de diversité.

- l'unité de production s'appelle le "cycle". Chaque ferme choisit plusieurs cycles dans **"cropping_cycles.csv"**
- l'unité de vente pour la ferme est le "lot" (**share**). Chaque cycle peut produire un certain nombre de lots en fonction du mois.
- l'unité d'achat pour le client est le "panier" composé d'un certain nombre de lots.
- La base de données **"cropping_cycles.csv"** contient plus de 1000 cycles de culture avec pour chaque cycle de culture:
  - un identifiant entier unique (`ID`), par exemple 233
  - le nom en français du légume concerné (`Crop_french`), par exemple `Tomate cerise`
  - la catégorie du légume concerné (`Crop category`), par exemple `Tomato`
  - le nombre de lots minimum par mois de récolte (`Shmin_month`) et le nombre de lot maximum total (`Shmax_total`) qu'on peut cultiver pour ce cycle
  - la durée de la récolte en mois (`Harvest_total`)
  - pour chaque mois de l'année, un booléen (0 ou 1) qui indique si le légume cultivé dans ce cycle peut être vendu (`Sale_Jan` , `Sale_Feb` , `Sale_Mar` ,...)

  Pour une micro-ferme, il s'agit donc de sélectionner les cycles qui vont lui permettre de répondre à la demande commerciale.

- La base de données **"demandes.csv"** qui contient les besoins de légumes dans les paniers hebdomadaires. En particuleir, il décrit pour chaque catégorie de légume:
  - le nom de la categorie (`Crop category`) (par exemple `Tomato`)
  - pour chaque mois, le nombre minimum de légumes différent qui doivent être proposées (`Minimal number of crops_Jan` , `Minimal number of crops_Feb` , ..., `Minimal number of crops_Dec`)
  - pour chaque mois, la quantité minimum en nombre de lots de cette catégoir de légume proposé dans cette catégorie (`Minimal quantity of shares_Jan` , `Minimal quantity of shares_Feb` ,..., `Minimal quantity of shares_Dec`)

Dans la base de données, plusieurs scénarios sont disponibles, ici on se restreindra à une stratégie de vente sur 12 mois (`Marketing` =12M) et à des climats modérés (`Climate` = `Mild`)

In [2]:
```
marketing="12M"
climat="Mild"
```

1- Prétraitement

1.1- Écrire `filtre_database` qui crée un dataframe pandas à partir d'un csv en filtrant sur les valeurs de marketing/climat

In [3]:
```
cycles=tme6.filtre_database("res/cropping_cycles.csv",marketing,climat)
cycles
```

Out[3]:

	Crop_french	ID	Seq	Prod_loc	Climate	Marketing	Settingup	Harvest_start	Harvest_end	Winter_storage	...	Growing_Mar	Growing_Apr	Growing_May	Growing_Jun	Growing_Jul	Growing_Aug	Gro
475	Tomate classique	579	NaN	Tunnel	Mild	12M	6	8	10	no	...	0	0	0	1	1	1	
476	Tomate classique	578	NaN	Tunnel	Mild	12M	5	7	10	no	...	0	0	1	1	1	1	
477	Tomate classique	577	NaN	Tunnel	Mild	12M	4	7	10	no	...	0	1	1	1	1	1	
478	Tomate classique	576	NaN	Tunnel	Mild	12M	4	6	10	no	...	0	1	1	1	1	1	
479	Tomate cerise	575	NaN	Tunnel	Mild	12M	5	7	10	no	...	0	0	1	1	1	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
758	Ail conservation	296	NaN	Tunnel	Mild	12M	1	6	6	yes	...	1	1	1	1	0	0	
759	Ail conservation	295	NaN	Tunnel	Mild	12M	10	5	5	yes	...	1	1	1	0	0	0	
760	Ail conservation	294	NaN	Field	Mild	12M	2	8	8	yes	...	1	1	1	1	1	1	
761	Ail conservation	293	NaN	Field	Mild	12M	11	6	6	yes	...	1	1	1	1	0	0	
762	Ail botte	292	NaN	Tunnel	Mild	12M	11	3	5	no	...	1	1	1	0	0	0	

288 rows × 41 columns

In [4]:
```
demands=tme6.filtre_database("res/demands.csv",marketing,climat)
demands
```

Out[4]:

	Crop category	Climate	Marketing	Minimal quantity of shares_Jan	Minimal quantity of shares_Feb	Minimal quantity of shares_Mar	Minimal quantity of shares_Apr	Minimal quantity of shares_May	Minimal quantity of shares_Jun	Minimal quantity of shares_Jul	...	Minimal number of crops_Jul	Minimal number of crops_Aug	Minimal number of crops_Sep	Minimal number of crops_Oct	Minimal number of crops_Nov	Minimal number of crops_Dec	Minima number o crops pe trimester_T
16	Carrot	Mild	12M	2	4	4	2	2	2	2	...	1	1	1	1	1	1	
18	Condiment crop	Mild	12M	2	4	4	4	4	4	4	...	2	2	2	2	2	2	
20	Fruit crop	Mild	12M	2	2	2	3	3	8	8	...	4	4	4	3	1	1	
22	Potato	Mild	12M	2	4	4	2	2	2	2	...	1	1	1	1	1	1	
24	Root crop	Mild	12M	2	4	4	4	4	1	1	...	1	1	1	2	2	2	
26	Tomato	Mild	12M	0	0	0	0	0	4	4	...	1	1	1	1	0	0	
28	Raw green	Mild	12M	2	4	4	4	4	4	4	...	1	1	1	1	1	1	
30	Cooked green	Mild	12M	2	4	4	4	4	1	1	...	2	2	2	2	2	2	

8 rows × 31 columns

1.2- Écrire une fonction `add_sale_duration(cycles)` qui ajoute dans `cycles` une colonne permettant de connaitre le nombre de total de mois de vente

Dans `cycles`, une série de colonne donne les mois de vente possible.

In [5]:
```
cycles[["Sale_"+i for i in tme6.months]]
```

Out[5]:

	Sale_Jan	Sale_Feb	Sale_Mar	Sale_Apr	Sale_May	Sale_Jun	Sale_Jul	Sale_Aug	Sale_Sep	Sale_Oct	Sale_Nov	Sale_Dec
475	0	0	0	0	0	0	0	1	1	1	0	0
476	0	0	0	0	0	0	1	1	1	1	0	0
477	0	0	0	0	0	0	1	1	1	1	0	0
478	0	0	0	0	0	1	1	1	1	1	0	0
479	0	0	0	0	0	0	1	1	1	1	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
758	1	1	1	1	0	1	1	1	1	1	1	1
759	1	1	1	1	1	1	1	1	1	1	1	1
760	1	1	1	1	0	0	0	1	1	1	1	1
761	1	1	1	1	0	1	1	1	1	1	1	1
762	0	0	1	1	1	0	0	0	0	0	0	0

288 rows × 12 columns

Mais nous n'avons pas de colonne nous indiquant le nombre total de mois de vente pour chaque cycle.

C'est le but de la fonction `add_sales_duration(cycles)` de créer une nouvelle `DataFrame`) à partir de `cycles` contenant cette colonne supplémentaire.

In [6]:
```
cycles2=tme6.add_sales_duration(cycles)
cycles2[["Sale_"+i for i in tme6.months]+["Sales_duration"]]
```

Out[6]:

	Sale_Jan	Sale_Feb	Sale_Mar	Sale_Apr	Sale_May	Sale_Jun	Sale_Jul	Sale_Aug	Sale_Sep	Sale_Oct	Sale_Nov	Sale_Dec	Sales_duration
475	0	0	0	0	0	0	0	1	1	1	0	0	3
476	0	0	0	0	0	0	1	1	1	1	0	0	4
477	0	0	0	0	0	0	1	1	1	1	0	0	4
478	0	0	0	0	0	1	1	1	1	1	0	0	5
479	0	0	0	0	0	0	1	1	1	1	0	0	4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
758	1	1	1	1	0	1	1	1	1	1	1	1	11
759	1	1	1	1	1	1	1	1	1	1	1	1	12
760	1	1	1	1	0	0	0	1	1	1	1	1	9
761	1	1	1	1	0	1	1	1	1	1	1	1	11
762	0	0	1	1	1	0	0	0	0	0	0	0	3

288 rows × 13 columns

2- Choix d'un cycle

Le choix d'un cycle se fait à partir d'une catégorie et d'un mois. Une **vraie** ferme utiliserait une stratégie ou un contexte pour sélectionner un cycle. Nous nous contenterons d'un choix aléatoire parmi les cycles possibles.

Il s'agit

- de vérifier qu'il y a une demande pour cette catégorie et ce mois (en vérifiant que `minimal number of crops` est >0)
- de sélectionner tous les cycles de cette catégorie et que l'on peut vendre ce mois
- puis de sélectionner au hasard l'un des cycles sélectionnés

2.1 Écrire la fonction `choose_cycle(cycles,demands,category,month)` qui rend l' `id` du cycle choisi ou `False` si elle a rencontré une impossibilité (pas de demandes ou pas de cycles sélectionnables)

In [7]:
```
id_chosen_cycle=tme6.choose_cycle(cycles2,demands,category = "Potato",month = "Mar")
# - vérifier dans demands si Potato est en demande en Mars : sinon False
# - sélectionner dans cycles2 si les cycles pour des légumes de la catégorie Potato en vente en Mars:
# - si aucun cycle trouvé : False
# - sinon, tirer au hasard un cycle

# (pour Potato/Mar :  choisir entre les cycles 514, 515, 516, 517, 520)
print("Quatres cycles au hasard\n-------------------------")
print(id_chosen_cycle)
print(tme6.choose_cycle(cycles2,demands,category = "Potato",month = "Mar"))
print(tme6.choose_cycle(cycles2,demands,category = "Potato",month = "Mar"))
print(tme6.choose_cycle(cycles2,demands,category = "Potato",month = "Mar"))
print("\n\n")
print("Cycle sélectionné : %d\n-------------------------"%id_chosen_cycle)
print(cycles2[cycles2["ID"]==id_chosen_cycle][["ID","Crop_french","Crop category","Sale_Mar"]])
```

```
Quatres cycles au hasard
--------------------------
516
517
514
514



Cycle sélectionné : 516
--------------------------
      ID                  Crop_french Crop category  Sale_Mar
538  516  Pomme de terre conservation        Potato         1
```

3- Choix d'une production mensualisée

Une fois les cycles choisis, nous savons que la production annuelle de chaque cycle est comprise entre 2 bornes qui dépendent de `Shmin_month`, `Harvest_total` et `Shmax_total` de ce cycle. Nous nous proposons de maintenir un dictionnaire `productions` qui associera, pour chaque `ID` de cycle, la production annuelle choisie.

In [8]:
```
productions=dict()
```

3.1- Proposer une fonction `update_production(productions,id_cycle,cycles)` qui met à jour `productions`

- Si le cycles n'existe pas dans le dataframe `cycles`, la fonction s'arrête et rend `False`
- Si ce cycle n'existe pas encore dans le dictionnaire `productions`, `productions` est initialisé pour ce cycle au minimum de sa production annuelle
- Si ce cycle existe, la production annuelle du cycle est incrémentée de 1 tnat qu'elle n'est pas arrivée au maximum possible.

La fonction rend `True` si il y a eu modification du dictionnaire `productions` et `False` sinon.

Par exemple, pour le cycle de `ID` 520, la production annuelle peut aller de $2*1$ à 4

In [9]:
```
print(cycles2[cycles2['ID'] == 520][["ID","Shmin_month","Harvest_total","Shmax_total"]])
```

```
     ID  Shmin_month  Harvest_total  Shmax_total
534  520            2              1            4
```

In [10]:
```
productions=dict() # re-init
print("productions=%s"%str(productions))
for i in range(5):
  print("#%d:"%i, end="")
  if tme6.update_production(productions,520,cycles2):
    print(" productions=%s"%str(productions))
  else:
    print(" erreur")
print("productions=%s"%str(productions))
```

```
productions={}
#0: productions={520: 2.0}
#1: productions={520: 3.0}
#2: productions={520: 4.0}
#3: erreur
#4: erreur
productions={520: 4.0}
```

Maintenant qu'on sait obtenir la production annuelle pour chaque cycle sélectionné, on peut construire un `agenda` qui, à partir du dictionnaire de productions et à partir de la dataframe des cycles fournit un dictionnaire qui, pour chaque mois, indique, pour chaque cycle, la quantité produite. On supposera que la production annuelle est répartie uniformément sur les mois de production du cycle.

3.2- Implémenter la fonction `create_agenda(productions,cycles)` qui fait ce travail.

In [11]:
```
# on crée notre programme de production annuelles (ATTENTION : résultat aléatoire)
productions=dict() # re-init

# idcycle = tme6.choose_cycle(cycles2,demands,category = "Carrot",month = "Jan"))
idcycle=337# choisi "au hasard avec la ligne ci-dessus : une carrote vendable en Janvier
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
print("productions=%s"%str(productions))

#idcycle=tme6.choose_cycle(cycles2,demands,category = "Potato",month = "Mar")
idcycle = 515 # choisi "au hasard avec la ligne ci-dessus : une pomme de terre vendable en Mars
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
print("productions=%s"%str(productions))

#idcycle=tme6.choose_cycle(cycles2,demands,category = "Potato",month = "Nov")
idcycle = 516 # choisi "au hasard avec la ligne ci-dessus : une pomme de terre vendable en November
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
print("productions=%s"%str(productions))

#idcycle=tme6.choose_cycle(cycles2,demands,category = "Fruit crop",month = "Apr")
idcycle=508 # choisi "au hasard avec la ligne ci-dessus : un fruit vendable en Avril
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
tme6.update_production(productions,idcycle,cycles2)
print("productions=%s"%str(productions))

print("Agenda : ")
agenda=tme6.create_agenda(productions,cycles2)
agenda
```

```
productions={337: 15.0}
productions={337: 15.0, 515: 3.0}
productions={337: 15.0, 515: 3.0, 516: 4.0}
productions={337: 15.0, 515: 3.0, 516: 4.0, 508: 4.0}
Agenda :
```

Out[11]:
```
{'Jan': {337: 2.5, 515: 0.375, 516: 0.5},
 'Feb': {337: 2.5, 515: 0.375, 516: 0.5},
 'Mar': {337: 2.5, 515: 0.375, 516: 0.5},
 'Apr': {337: 2.5, 515: 0.375, 516: 0.5, 508: 2.0},
 'May': {508: 2.0},
 'Jun': {},
 'Jul': {},
 'Aug': {},
 'Sep': {515: 0.375, 516: 0.5},
 'Oct': {515: 0.375, 516: 0.5},
 'Nov': {337: 2.5, 515: 0.375, 516: 0.5},
 'Dec': {337: 2.5, 515: 0.375, 516: 0.5}}
```

4- construction d'une micro-ferme valide

Nous savons donc proposer un agenda pour une microferme qui pourra constituer un panier chaque mois avec ses cycles. Mais ces paniers doivent vérifier certains critères (on voit par exemple que notre ferme ci-dessus n'a pas du tout de panier pour les mois de Mai à Juillet).

Les critères sont décrits dans `demands` : il faut une certaine quantité et une certain diversité :

- **quantité** par mois et par catégorie, `demands` fournit une "Minimal quantity of shares" que doit vérifier la qté produites ce mois et pour cette catégorie
- **diversité** par mois et par catégorie, le nombre de cycles (ou crops) doit dépasser chaque mois un "Minimal number of crops"

4.1- Implémenter la fonction `check_qty(agenda,cycle,demands)` qui renvoie la liste des couples `(mois,catégorie)` qui ne respectent pas cette contrainte

Pour chaque mois, pour chaque catégorie dans `demands` , et si `Minimal quantity of shares>0` , vérifier que la somme des productions de tous les légumes de cette catégorie dans ce mois dépasse ce minimum et retourner la liste des défauts.

In [12]:
```
print(tme6.check_qty(agenda,cycles2,demands))
```

```
[('Carrot', 'Feb'), ('Carrot', 'Mar'), ('Carrot', 'May'), ('Carrot', 'Jun'), ('Carrot', 'Jul'), ('Carrot', 'Aug'), ('Carrot', 'Sep'), ('Carrot', 'Oct'), ('Carrot', 'Dec'), ('Condiment cro
p', 'Jan'), ('Condiment crop', 'Feb'), ('Condiment crop', 'Mar'), ('Condiment crop', 'Apr'), ('Condiment crop', 'May'), ('Condiment crop', 'Jun'), ('Condiment crop', 'Jul'), ('Condiment c
rop', 'Aug'), ('Condiment crop', 'Sep'), ('Condiment crop', 'Oct'), ('Condiment crop', 'Nov'), ('Condiment crop', 'Dec'), ('Fruit crop', 'Jan'), ('Fruit crop', 'Feb'), ('Fruit crop', 'Ma
r'), ('Fruit crop', 'Apr'), ('Fruit crop', 'May'), ('Fruit crop', 'Jun'), ('Fruit crop', 'Jul'), ('Fruit crop', 'Aug'), ('Fruit crop', 'Sep'), ('Fruit crop', 'Oct'), ('Fruit crop', 'No
v'), ('Fruit crop', 'Dec'), ('Potato', 'Jan'), ('Potato', 'Feb'), ('Potato', 'Mar'), ('Potato', 'Apr'), ('Potato', 'May'), ('Potato', 'Jun'), ('Potato', 'Jul'), ('Potato', 'Aug'), ('Potat
o', 'Sep'), ('Potato', 'Oct'), ('Potato', 'Nov'), ('Potato', 'Dec'), ('Root crop', 'Jan'), ('Root crop', 'Feb'), ('Root crop', 'Mar'), ('Root crop', 'Apr'), ('Root crop', 'May'), ('Root c
rop', 'Jun'), ('Root crop', 'Jul'), ('Root crop', 'Aug'), ('Root crop', 'Sep'), ('Root crop', 'Oct'), ('Root crop', 'Nov'), ('Root crop', 'Dec'), ('Tomato', 'Jun'), ('Tomato', 'Jul'), ('T
omato', 'Aug'), ('Tomato', 'Sep'), ('Tomato', 'Oct'), ('Raw green', 'Jan'), ('Raw green', 'Feb'), ('Raw green', 'Mar'), ('Raw green', 'Apr'), ('Raw green', 'May'), ('Raw green', 'Jun'),
('Raw green', 'Jul'), ('Raw green', 'Aug'), ('Raw green', 'Sep'), ('Raw green', 'Oct'), ('Raw green', 'Nov'), ('Raw green', 'Dec'), ('Cooked green', 'Jan'), ('Cooked green', 'Feb'), ('Coo
ked green', 'Mar'), ('Cooked green', 'Apr'), ('Cooked green', 'May'), ('Cooked green', 'Jun'), ('Cooked green', 'Jul'), ('Cooked green', 'Aug'), ('Cooked green', 'Sep'), ('Cooked green',
'Oct'), ('Cooked green', 'Nov'), ('Cooked green', 'Dec')]
```

4.2- implémenter la fonction `check_diversity(agenda,cycles,demands)` qui renvoie pour chaque mois, le nombre de mois où il n'y a pas assez de légumes différents par catégorie

Pour chaque mois, et pour chaque catégorie dans `demands` , vérifier si `Minimal number of crops` est plus petit que le nombre de légumes de cette catégorie et retourner la liste des défauts.

In [13]:
```
print(tme6.check_diversity(agenda,cycles2,demands))
```

```
[('Carrot', 'May'), ('Carrot', 'Jun'), ('Carrot', 'Jul'), ('Carrot', 'Aug'), ('Carrot', 'Sep'), ('Carrot', 'Oct'), ('Condiment crop', 'Jan'), ('Condiment crop', 'Feb'), ('Condiment crop',
'Mar'), ('Condiment crop', 'Apr'), ('Condiment crop', 'May'), ('Condiment crop', 'Jun'), ('Condiment crop', 'Jul'), ('Condiment crop', 'Aug'), ('Condiment crop', 'Sep'), ('Condiment cro
p', 'Oct'), ('Condiment crop', 'Nov'), ('Condiment crop', 'Dec'), ('Fruit crop', 'Jan'), ('Fruit crop', 'Feb'), ('Fruit crop', 'Mar'), ('Fruit crop', 'Apr'), ('Fruit crop', 'May'), ('Frui
t crop', 'Jun'), ('Fruit crop', 'Jul'), ('Fruit crop', 'Aug'), ('Fruit crop', 'Sep'), ('Fruit crop', 'Oct'), ('Fruit crop', 'Nov'), ('Fruit crop', 'Dec'), ('Potato', 'May'), ('Potato', 'J
un'), ('Potato', 'Jul'), ('Potato', 'Aug'), ('Root crop', 'Jan'), ('Root crop', 'Feb'), ('Root crop', 'Mar'), ('Root crop', 'Apr'), ('Root crop', 'May'), ('Root crop', 'Jun'), ('Root cro
p', 'Jul'), ('Root crop', 'Aug'), ('Root crop', 'Sep'), ('Root crop', 'Oct'), ('Root crop', 'Nov'), ('Root crop', 'Dec'), ('Tomato', 'Jun'), ('Tomato', 'Jul'), ('Tomato', 'Aug'), ('Tomat
o', 'Sep'), ('Tomato', 'Oct'), ('Raw green', 'Jan'), ('Raw green', 'Feb'), ('Raw green', 'Mar'), ('Raw green', 'Apr'), ('Raw green', 'May'), ('Raw green', 'Jun'), ('Raw green', 'Jul'),
('Raw green', 'Aug'), ('Raw green', 'Sep'), ('Raw green', 'Oct'), ('Raw green', 'Nov'), ('Raw green', 'Dec'), ('Cooked green', 'Jan'), ('Cooked green', 'Feb'), ('Cooked green', 'Mar'),
('Cooked green', 'Apr'), ('Cooked green', 'May'), ('Cooked green', 'Jun'), ('Cooked green', 'Jul'), ('Cooked green', 'Aug'), ('Cooked green', 'Sep'), ('Cooked green', 'Oct'), ('Cooked gre
en', 'Nov'), ('Cooked green', 'Dec')]
```

4.3- Génération d'une microferme valide : `generate_microferme(cycles,demands)`

Nous sommes enfin prêt à pouvoir générer une valide : en partant d'une production vide, tant qu'une des 2 contraintes n'est pas vérifiée, elles nous fournissent une liste de couples (mois,catégorie), on choisit un de couples au hasard, et on `update_production` en choisissant un légume au hasard avec `choose_cycle` . La fonction nous retourne la productions prévues pour cette microferme.

Astuce : quand vous rajoutez un nouveau couple (mois,catégorie), appelez `choose_cycle` un grand nombre de fois (par exemple, entre une 30aine et une 60aine de fois, au hasard) avec les mêmes arguments pour ne pas rester au minimum de production de chaque cycle.

In [14]:
```
%%time
mf=tme6.generate_microferme(cycles2,demands)
```

```
CPU times: user 5.44 s, sys: 76.1 ms, total: 5.52 s
Wall time: 5.51 s
```

In [15]:
```
print(mf)
```

```
{462: 20.0, 559: 8.0, 508: 8.0, 516: 32.0, 505: 8.0, 299: 20.0, 390: 32.0, 402: 28.0, 328: 8.0, 481: 8.0, 523: 8.0, 452: 8.0, 526: 4.0, 338: 24.0, 518: 12.0, 478: 8.0, 330: 16.0, 415: 16.
0, 342: 32.0, 570: 12.0, 496: 40.0, 575: 16.0, 566: 8.0, 414: 12.0, 574: 20.0, 407: 4.0, 305: 12.0, 519: 12.0, 386: 12.0, 403: 48.0, 406: 8.0, 382: 16.0, 430: 12.0, 556: 8.0, 426: 20.0, 4
83: 28.0, 488: 8.0, 319: 8.0, 507: 8.0, 322: 8.0, 477: 8.0, 363: 8.0, 454: 8.0, 377: 8.0, 418: 8.0, 359: 8.0, 306: 4.0, 362: 8.0}
```

4.3- Simulation d'un grand nombre de microfermes valides

Cette fonction prend du temps... Et il nous faut beaucoup de simulations de microfermes (des milliers ...). Nous allons donc créer un tableau de N microfermes et le sauvegarder dans un fichier json :

In [16]:
```
%%time
N=10
fermes=[tme6.generate_microferme(cycles2,demands) for _ in range(N)]
```

```
CPU times: user 56 s, sys: 642 ms, total: 56.6 s
Wall time: 56.5 s
```

In [17]:
```
import json
with open("fermes.json", "w") as outfile:
    json.dump(fermes, outfile, indent=2)
```

In [18]:
```
print(fermes)
```

[{296: 44.0, 390: 32.0, 519: 12.0, 348: 8.0, 333: 28.0, 477: 8.0, 312: 12.0, 328: 8.0, 578: 16.0, 521: 8.0, 344: 28.0, 330: 16.0, 396: 12.0, 458: 16.0, 460: 12.0, 400: 40.0, 298: 20.0, 308: 20.0, 305: 12.0, 507: 8.0, 517: 28.0, 314: 8.0, 318: 32.0, 304: 8.0, 514: 32.0, 422: 12.0, 473: 16.0, 410: 8.0, 565: 4.0, 389: 32.0, 363: 8.0, 354: 4.0, 574: 20.0, 549: 4.0, 392: 16.0, 432: 4.0, 467: 12.0, 504: 8.0, 512: 16.0, 377: 8.0, 347: 12.0, 320: 8.0, 418: 8.0, 379: 12.0, 375: 8.0, 307: 16.0, 295: 43.0, 427: 12.0}, {297: 12.0, 419: 4.0, 339: 24.0, 330: 16.0, 490: 8.0, 575: 16.0, 535: 4.0, 305: 12.0, 389: 32.0, 516: 32.0, 519: 12.0, 459: 16.0, 509: 4.0, 492: 36.0, 415: 16.0, 548: 8.0, 462: 20.0, 384: 12.0, 386: 12.0, 576: 20.0, 363: 8.0, 293: 42.0, 312: 12.0, 428: 4.0, 510: 12.0, 524: 4.0, 395: 16.0, 410: 8.0, 329: 4.0, 311: 4.0, 494: 28.0, 450: 8.0, 477: 8.0, 498: 16.0, 422: 12.0, 318: 32.0, 426: 20.0, 326: 8.0, 557: 4.0, 307: 16.0, 350: 8.0, 503: 16.0, 508: 8.0, 315: 8.0, 378: 8.0, 325: 8.0, 292: 12.0}, {400: 40.0, 367: 4.0, 457: 8.0, 353: 4.0, 348: 8.0, 486: 24.0, 390: 32.0, 313: 8.0, 340: 4.0, 454: 8.0, 492: 32.0, 519: 12.0, 474: 16.0, 429: 12.0, 423: 8.0, 540: 32.0, 516: 32.0, 330: 16.0, 506: 8.0, 512: 16.0, 339: 24.0, 477: 8.0, 377: 8.0, 499: 12.0, 403: 45.0, 576: 20.0, 328: 8.0, 480: 8.0, 551: 4.0, 410: 8.0, 471: 12.0, 374: 12.0, 497: 16.0, 296: 39.0, 490: 8.0, 347: 12.0, 435: 12.0, 319: 8.0, 566: 8.0, 389: 32.0, 547: 4.0, 321: 16.0, 427: 12.0, 342: 32.0, 521: 8.0, 416: 8.0, 365: 8.0, 396: 12.0, 408: 8.0, 502: 16.0}, {477: 8.0, 449: 8.0, 387: 36.0, 498: 16.0, 515: 30.0, 508: 8.0, 551: 4.0, 377: 8.0, 519: 12.0, 491: 40.0, 550: 4.0, 495: 24.0, 338: 24.0, 465: 12.0, 296: 44.0, 330: 16.0, 313: 8.0, 459: 16.0, 426: 20.0, 318: 32.0, 467: 12.0, 524: 4.0, 351: 12.0, 437: 4.0, 298: 20.0, 413: 4.0, 503: 16.0, 383: 16.0, 305: 12.0, 421: 4.0, 388: 36.0, 568: 20.0, 480: 8.0, 328: 8.0, 553: 4.0, 454: 8.0, 325: 8.0, 510: 12.0, 308: 20.0, 517: 28.0, 488: 8.0, 540: 32.0, 397: 20.0}, {514: 32.0, 312: 12.0, 477: 8.0, 322: 8.0, 426: 20.0, 469: 8.0, 298: 20.0, 366: 12.0, 307: 16.0, 401: 35.0, 343: 28.0, 387: 36.0, 440: 8.0, 519: 12.0, 347: 12.0, 447: 4.0, 575: 16.0, 384: 12.0, 403: 48.0, 308: 20.0, 338: 24.0, 546: 8.0, 330: 16.0, 386: 12.0, 391: 8.0, 498: 16.0, 521: 8.0, 422: 12.0, 383: 16.0, 576: 20.0, 325: 8.0, 473: 16.0, 533: 8.0, 467: 12.0, 381: 12.0, 326: 8.0, 377: 8.0, 468: 12.0, 317: 36.0, 315: 8.0, 328: 8.0, 305: 12.0, 433: 8.0, 389: 32.0}, {304: 8.0, 414: 12.0, 546: 8.0, 326: 8.0, 515: 32.0, 492: 36.0, 518: 12.0, 312: 12.0, 512: 16.0, 430: 12.0, 370: 8.0, 474: 16.0, 466: 12.0, 377: 8.0, 339: 24.0, 494: 28.0, 425: 16.0, 347: 12.0, 293: 44.0, 342: 32.0, 391: 8.0, 575: 16.0, 330: 16.0, 503: 16.0, 310: 8.0, 408: 8.0, 399: 39.0, 328: 8.0, 374: 12.0, 519: 12.0, 305: 12.0, 380: 8.0, 436: 8.0, 576: 20.0, 446: 4.0, 381: 12.0, 320: 8.0, 388: 36.0, 298: 20.0, 358: 8.0, 564: 4.0, 314: 8.0, 308: 20.0, 306: 4.0, 393: 16.0, 321: 16.0, 426: 20.0, 392: 16.0, 490: 8.0}, {485: 24.0, 515: 32.0, 384: 12.0, 570: 12.0, 296: 44.0, 486: 24.0, 336: 24.0, 552: 8.0, 316: 4.0, 298: 20.0, 459: 16.0, 377: 8.0, 464: 12.0, 572: 12.0, 330: 16.0, 372: 4.0, 473: 16.0, 293: 31.0, 576: 20.0, 398: 12.0, 317: 36.0, 389: 32.0, 450: 8.0, 427: 12.0, 356: 20.0, 397: 20.0, 523: 8.0, 519: 12.0, 310: 8.0, 366: 12.0, 500: 12.0, 343: 28.0, 295: 43.0, 368: 16.0, 462: 20.0, 328: 8.0, 362: 8.0, 359: 8.0, 406: 8.0, 477: 8.0, 418: 8.0, 506: 8.0, 320: 8.0, 433: 8.0, 360: 8.0, 508: 8.0, 393: 16.0}, {308: 20.0, 401: 36.0, 484: 28.0, 312: 12.0, 450: 8.0, 476: 8.0, 334: 28.0, 358: 8.0, 517: 28.0, 571: 16.0, 345: 24.0, 332: 8.0, 465: 12.0, 516: 32.0, 326: 8.0, 368: 16.0, 298: 20.0, 347: 12.0, 426: 20.0, 399: 39.0, 389: 32.0, 545: 4.0, 427: 12.0, 458: 16.0, 329: 4.0, 529: 4.0, 431: 8.0, 309: 16.0, 330: 16.0, 397: 20.0, 570: 12.0, 344: 28.0, 503: 16.0, 519: 12.0, 523: 8.0, 576: 20.0, 351: 12.0, 360: 8.0, 375: 8.0, 396: 12.0, 418: 8.0, 508: 8.0, 488: 8.0, 408: 8.0, 482: 4.0, 404: 12.0, 377: 8.0, 366: 12.0}, {336: 24.0, 477: 8.0, 312: 12.0, 578: 16.0, 514: 32.0, 429: 12.0, 330: 16.0, 523: 8.0, 573: 20.0, 447: 4.0, 387: 36.0, 518: 12.0, 344: 28.0, 294: 36.0, 293: 44.0, 378: 8.0, 453: 8.0, 308: 20.0, 503: 16.0, 410: 8.0, 425: 16.0, 497: 16.0, 469: 8.0, 298: 20.0, 326: 8.0, 519: 12.0, 454: 8.0, 319: 8.0, 366: 12.0, 328: 8.0, 397: 20.0, 343: 28.0, 462: 20.0, 488: 8.0, 466: 12.0, 374: 12.0, 451: 12.0, 386: 12.0, 408: 8.0, 506: 8.0, 468: 12.0, 321: 16.0, 481: 8.0}, {331: 4.0, 474: 16.0, 519: 12.0, 433: 8.0, 516: 32.0, 459: 16.0, 347: 12.0, 305: 12.0, 575: 16.0, 511: 16.0, 335: 28.0, 472: 12.0, 481: 8.0, 389: 32.0, 493: 32.0, 456: 8.0, 535: 4.0, 375: 8.0, 399: 33.0, 317: 36.0, 329: 4.0, 425: 16.0, 369: 4.0, 353: 4.0, 309: 16.0, 328: 8.0, 298: 20.0, 330: 16.0, 307: 16.0, 439: 8.0, 409: 4.0, 351: 12.0, 393: 16.0, 504: 8.0, 312: 12.0, 521: 8.0, 344: 28.0, 391: 8.0, 502: 16.0, 503: 16.0, 420: 4.0, 358: 8.0, 397: 20.0, 403: 48.0, 408: 8.0, 568: 20.0, 480: 8.0}]

In [ ]: